

Ważenie strat w uczeniu wielozadaniowym (MTL)

Przegląd metod dla projektu ADMET-predictor — mieszane zadania regresji i klasyfikacji.

1. Po co to czytasz

W uczeniu wielozadaniowym (Multi-Task Learning, MTL) jeden model uczony jest jednocześnie kilku różnych zadań, dzieląc wspólną reprezentację (encoder), a różniąc się tylko głowicami predykcyjnymi (po jednej na zadanie). Dzięki temu wiedza nauczona z jednego endpointu może wspomóc inny — szczególnie gdy zadania są biologicznie powiązane i dane treningowe są ograniczone.

Problem: gdy łączymy zadania regresji i klasyfikacji w jednej sieci, ich surowe wartości strat mają zupełnie różne skale. MSE na nieprzeskalowanej regresji może być rzędu setek, a BCE na klasyfikacji to zwykle wartości w przedziale 0.3–1.0. Jeśli zsumujemy je naiwnie ($L = L_{\text{reg}} + L_{\text{cls}}$), regresja zdominuje gradient i klasyfikacja praktycznie się nie nauczy — lub odwrotnie, jeśli mamy dużo zadań klasyfikacyjnych a jedną regresję.

Drugi problem to liczba zadań: jeśli mamy 3 zadania regresji i 1 klasyfikacji, suma daje 3× większy gradient regresji niż klasyfikacji, niezależnie od skali. To psuje balans treningu.

Poniższy dokument streszcza najważniejsze podejścia do ważenia strat w MTL, od najprostszych baseline’ów po nowoczesne metody adaptacyjne, z odniesieniami do oryginalnych publikacji.

2. Przegląd metod

1. Sum (suma nieważona) — baseline

Najprostsze podejście: $L = \sum L_i$. Wszystkie straty sumowane bez wag. Działa tylko gdy etykiety regresji są wcześniej przeskalowane (np. `StandardScaler`) tak, by MSE miało rząd wielkości porównywalny z BCE. W innych przypadkach większa skala dominuje. Wrażliwe na liczbę zadań — im więcej zadań tego samego typu, tym większy ich łączny wpływ.

□ Źródło: Baseline, używane jako punkt odniesienia w większości publikacji MTL.

2. Uniform average (średnia po zadaniach)

$L = \text{mean}(L_i)$ — każde zadanie ma równy wpływ niezależnie od ich liczby. Naprawia problem „3 zadania regresji dominują 1 klasyfikację” (z sumy do średniej). Nadal wymaga, by skale strat były podobne. Bardzo dobry punkt startowy — jedna linia zmiany w kodzie.

□ Źródło: Standardowy baseline, brak konkretnego „papera autorskiego”.

3. Static weights (stałe wagi ręczne)

$L = \sum w_i L_i$ z wagami w_i dobranymi ręcznie (np. przez grid-search albo intuicyjnie: $w_{\text{cls}} \approx 5\text{--}10 \times w_{\text{reg}}$, jeśli BCE jest kilkukrotnie mniejszy od MSE). Proste, ale wymaga eksperymentacji i nie skaluje się do wielu zadań.

□ Źródło: Wymieniane jako baseline w (Kendall et al., 2018).

4. Uncertainty Weighting — Kendall, Gal & Cipolla (2018) □

Każde zadanie dostaje uczony parametr σ_i reprezentujący „niepewność homoskedastyczną”. Funkcja straty:

$$L = \sum (1 / 2\sigma_i^2) \cdot L_i + \log \sigma_i$$

Pierwszy człon to ważona strata, drugi ($\log \sigma$) jest regularizatorem zapobiegającym $\sigma \rightarrow \infty$. Sieć sama uczy się wag — zadania zaszumione/trudne dostają niższe wagi, łatwe wyższe. Idealne dla mieszanych zadań regresji i klasyfikacji: oryginalny paper demonstuje to dokładnie na takim przypadku (regresja głębokości + klasyfikacja semantyczna). Implementacja to ok. 10 linii kodu (`nn.ParameterDict` z parametrami `log_var`).

□ Źródło: A. Kendall, Y. Gal, R. Cipolla. Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics. CVPR 2018. arXiv:1705.07115.

5. GradNorm — Chen et al. (2018)

Dynamicznie dostosowuje wagi tak, by normy gradientów wszystkich zadań w warstwie wspólnej (encoderze) były podobnej wielkości. Wagi są aktualizowane przez osobny optymalizator na podstawie tempa uczenia każdego zadania. Wymaga podpięcia gradient hooków i jest bardziej skomplikowane w implementacji niż uncertainty weighting, ale skuteczne gdy gradienty zadań różnią się rzędami wielkości.

□ Źródło: Z. Chen, V. Badrinarayanan, C.-Y. Lee, A. Rabinovich. GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks. ICML 2018. arXiv:1711.02257.

6. Dynamic Weight Average (DWA) — Liu, Johns & Davison (2019)

Wagi proporcjonalne do tempa spadku straty w ostatnich epokach. Zadania uczące się wolniej (mniejszy spadek L) dostają większą wagę. Działa post-hoc na samych wartościach loss (bez gradient hooków), więc znacznie prostsze niż GradNorm. Dobre do mieszanych zadań, bo automatycznie wyrównuje tempo uczenia niezależnie od skali.

□ Źródło: S. Liu, E. Johns, A. J. Davison. End-to-End Multi-Task Learning with Attention. CVPR 2019. arXiv:1803.10704.

7. MGDA — Sener & Koltun (2018)

Traktuje MTL jako problem optymalizacji wielokryterialnej (multi-objective). Szuka kierunku spadku gradientu, który jest wspólny dla wszystkich zadań (rozwiązanie Pareto-optymalne) za pomocą algorytmu Frank–Wolfe. Solidny teoretycznie, ale obliczeniowo kosztowny — często overkill dla projektów z małą liczbą zadań.

□ Źródło: O. Sener, V. Koltun. Multi-Task Learning as Multi-Objective Optimization. NeurIPS 2018. arXiv:1810.04650.

8. PCGrad (Gradient Surgery) — Yu et al. (2020)

Gdy gradienty dwóch zadań są w konflikcie (ujemny iloczyn skalarny), PCGrad rzutuje jeden gradient na płaszczyznę ortogonalną do drugiego — usuwa „szkodliwą” komponentę. Pomocne, gdy podejrzewamy transfer negatywny (np. AMES + Caco-2 w naszym projekcie). Wymaga manualnej manipulacji gradientami w pętli treningowej.

□ Źródło: T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, C. Finn. Gradient Surgery for Multi-Task Learning. NeurIPS 2020. arXiv:2001.06782.

9. CAGrad — Liu et al. (2021)

Rozwinięcie PCGrad: szuka kierunku, który nie pogarsza żadnego zadania (conflict-averse gradient descent). Bardziej principled teoretycznie niż PCGrad, ale rzadziej implementowane. Polecane jeśli dane mają silne konflikty między zadaniami.

□ Źródło: B. Liu, X. Liu, X. Jin, P. Stone, Q. Liu. Conflict-Averse Gradient Descent for Multi-Task Learning. NeurIPS 2021. arXiv:2110.14048.

3. Pozycje ogólne / przeglądowe

Jeśli potrzebujesz cytować coś jako przegląd całej dziedziny MTL (np. w pracy dyplomowej):

- R. Caruana. Multitask Learning. Machine Learning 28(1), 1997. — klasyk, źródło samego pojęcia MTL.
- S. Vandenhende, S. Georgoulis, W. Van Gansbeke, M. Proesmans, D. Dai, L. Van Gool. Multi-Task Learning for Dense Prediction Tasks: A Survey. IEEE TPAMI 2021. arXiv:2004.13379. — nowoczesny przegląd z dedykowaną sekcją o loss balancing.
- M. Crawshaw. Multi-Task Learning with Deep Neural Networks: A Survey. 2020. arXiv:2009.09796. — ogólny survey, bardzo czytelny dla osób wchodzących w temat.

4. Rekomendacja dla projektu ADMET-predictor

Plan minimalny → ambitny:

Krok 1 (must-have). Zacząć od Uniform Average (metoda #2) — jedna linia zmiany w pętli treningowej (`torch.stack(losses).mean()` zamiast sumy). Od razu naprawia „3 reg vs 1 cls”.

Krok 2 (główny eksperyment). Dorzucić Uncertainty Weighting wg Kendall et al. (metoda #4). Implementacja: dodać `nn.ParameterDict({task: nn.Parameter(torch.zeros(1)) for task in all_tasks})` do modelu, w pętli treningowej obliczyć `weighted = exp(-log_var) * raw_loss + 0.5 * log_var`. Pokazać na slajdzie tabelkę „equal vs uncertainty” — ładny wkład eksperymentalny.

Krok 3 (opcjonalnie). DWA (metoda #6) jako trzeci wariant porównawczy. Też prosty (post-hoc na wartościach loss), uzupełnia dyskusję wyników.

Co pominąć. GradNorm, MGDA, PCGrad, CAGrad — ciekawe i nowoczesne, ale wymagają dużo inżynierii (gradient hooks, custom backward) i wnoszą nieproporcjonalnie mało wartości w projekcie studenckim. Warto je tylko wspomnieć w pracy jako kierunki rozwoju.